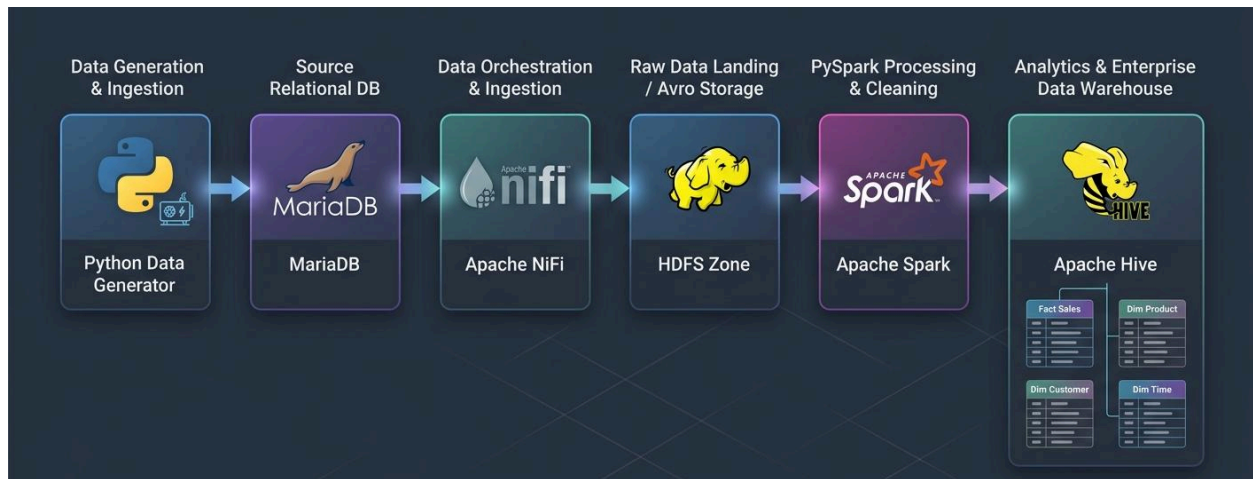


{ Data Engineering ETL Pipeline Task }

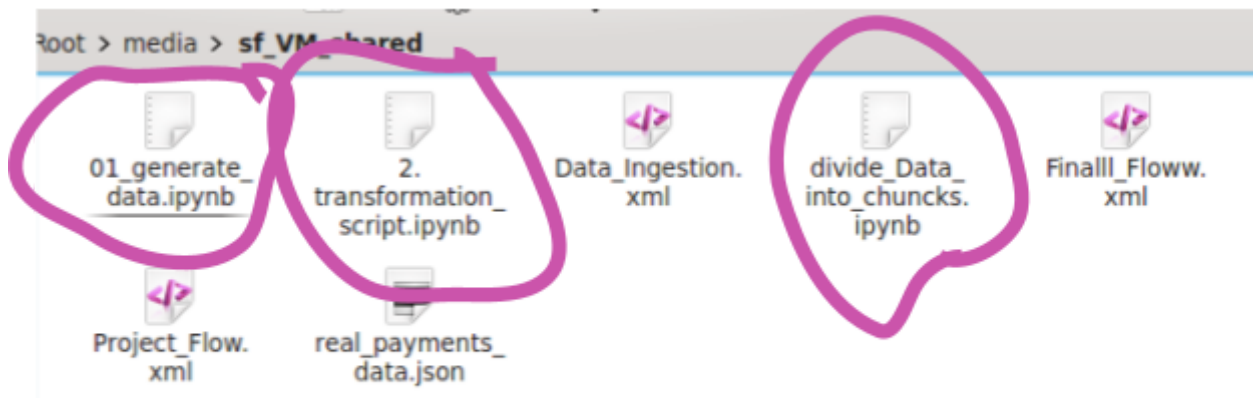
Github Project

Architecture diagram



1. Generate Data Using Python

First i put python file script to VM_shared folder and upload to pyspark jupyter



Then open pyspark

```
[student@localhost ~]$ pyspark
[I 20:34:32.146 NotebookApp] Serving notebooks from local directory: /home/student
[I 20:34:32.146 NotebookApp] Jupyter Notebook 6.4.3 is running at:
[I 20:34:32.146 NotebookApp] http://localhost:3333/?token=7f38e241991acf8fd8c5ad1ac6d8a7e0c075913ecd983b9a
[I 20:34:32.146 NotebookApp] or http://127.0.0.1:3333/?token=7f38e241991acf8fd8c5ad1ac6d8a7e0c075913ecd983b9a
[I 20:34:32.146 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 20:34:32.175 NotebookApp]

To access the notebook, open this file in a browser:
file:///home/student/.local/share/jupyter/runtime/nbserver-4643-open.html
```



First problem i face the sql connector didn't exist



```
In [2]: import mysql.connector
        from faker import Faker
        import random
        from datetime import datetime, timedelta

-----
ModuleNotFoundError                               Traceback (most recent call last)
/tmp/ipykernel_4920/3512011017.py in <module>
----> 1 import mysql.connector
      2 from faker import Faker
      3 import random
      4 from datetime import datetime, timedelta

ModuleNotFoundError: No module named 'mysql'
```

So i use **pip install mysql-connector-python faker** to install it

```
[student@localhost ~]$ !pip install mysql-connector-python faker
bash: !pip: event not found
[student@localhost ~]$ pip install mysql-connector-python faker
Defaulting to user installation because normal site-packages is not writeable
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.0.0-cp38-cp38-manylinux_2_17_x86_64.whl (19.3 MB)
    |#####| 19.3 MB 1.8 MB/s
Requirement already satisfied: faker in /usr/local/lib/python3.8/site-packages (8.12.1)
Requirement already satisfied: python-dateutil>=2.4 in /usr/local/lib/python3.8/site-packages (from faker)
Requirement already satisfied: text-unidecode==1.3 in /usr/local/lib/python3.8/site-packages (from faker) (
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.8/site-packages (from python-dateutil>=2.
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.0.0
WARNING: You are using pip version 21.2.4; however, version 25.0.1 is available.
You should consider upgrading via the '/usr/local/bin/python3.8 -m pip install --upgrade pip' command.
```

About configuration the instructor used it's for system configurations, making the code easily configurable without altering the main execution logic:

- **Database Connection Credentials (DB_*)**: Specifies the parameters required to connect to the MySQL database instance, including host (**localhost**), port (**3306**), user credentials, and the target database name (**ecommerce**).
- **Data Volume Parameters (*_COUNT)**: Sets the target number of records to be generated for each table (e.g., 100k customers, 500k orders, and 2M order items) to simulate a realistic e-commerce workload.
- **Batch Processing Size (BATCH_SIZE)**: Defines the chunk size (5,000 records) for batch insertion into MySQL, optimizing execution performance and preventing memory overhead during bulk operations.

- **Faker Initialization (`fake = Faker()`):** Instantiates the **Faker** generator object to create realistic synthetic data (e.g., names, addresses, emails).

```
# =====
# CONFIGURATION
# =====

DB_HOST = "localhost"
DB_PORT = 3306
DB_USER = "student"
DB_PASSWORD = "student"
DB_NAME = "ecommerce"

CUSTOMERS_COUNT = 100_000
PRODUCTS_COUNT = 20_000
ORDERS_COUNT = 500_000
ORDER_ITEMS_COUNT = 2_000_000
PAYMENTS_COUNT = 500_000

BATCH_SIZE = 5_000

fake = Faker()
```

2. Store Data in MariaDB

After i run all file generate_data it's already save data into MariaDB and go to CI to make sure it's already saved by using **mysql -u student -p** then write password

Or **mysql - -user=student - -password=student**

```
[student@localhost ~]$ mysql -u student -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 5.5.68-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE ecommerce;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [ecommerce]> SHOW TABLES;
+-----+
| Tables_in_ecommerce |
+-----+
| customers            |
| order_items          |
| orders               |
| payments             |
| products             |
+-----+
5 rows in set (0.00 sec)
```

```

MariaDB [ecommerce]> SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
-> UNION ALL
-> SELECT 'products', COUNT(*) FROM products
-> UNION ALL
-> SELECT 'orders', COUNT(*) FROM orders
-> UNION ALL
-> SELECT 'order_items', COUNT(*) FROM order_items
-> UNION ALL
-> SELECT 'payments', COUNT(*) FROM payments;
+-----+-----+
| table_name | row_count |
+-----+-----+
| customers  | 100000    |
| products   | 20000     |
| orders     | 500000    |
| order_items | 2000000   |
| payments   | 500000    |
+-----+-----+
5 rows in set (0.42 sec)

```

Then i upload jar file to connect MariaDB with other tools used java language like Nifi , spark . DBreaver

```

[student@localhost ~]$
[student@localhost ~]$ cd /media/sf VM_shared
[student@localhost sf_VM_shared]$ ls
01_generate_data.ipynb      Data_Ingestion.xml          Finalll_Floww.xml          Project_Flow.xml
2.transformation_script.ipynb  divide_Data_into_chuncks.ipynb  mariadb-java-client-3.5.10.jar  real_payments_data.json
[student@localhost sf_VM_shared]$ sudo cp mariadb-java-client-3.5.10.jar /usr/local/nifi/nifi-1.14.0/lib/
[student@localhost sf_VM_shared]$ ls -l /usr/local/nifi/nifi-1.14.0/lib/mariadb-java-client-3.5.10.jar
-rwxr-x--- 1 root root 791739 Sep  6 22:20 /usr/local/nifi/nifi-1.14.0/lib/mariadb-java-client-3.5.10.jar
[student@localhost sf_VM_shared]$

```

3. Extract Data Using Apache NiFi

Used first processor to just gave him database and he read all tables dynamically discovers and fetches metadata for all existing tables in the MariaDB source database (ecommerce).

	ListDatabaseTables ListDatabaseTables 1.14.0 org.apache.nifi - nifi-standard-nar	 1
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Ingestion Mechanism: Uses `DBCPConnectionPool` JDBC Controller Service to establish a stable connection with the relational database.

RunningSTOP & CONFIGURE

SETTINGSSCHEDULINGPROPERTIESCOMMENTS

Required field

Property	Value
Database Connection Pooling Service	 <code>DBCPConnectionPool</code>
Catalog	 <code>ecommerce</code>
Schema Pattern	 No value set
Table Name Pattern	 No value set
Table Types	 <code>TABLE</code>
Include Count	 <code>false</code>
Record Writer	 No value set
Refresh Interval	 <code>0 sec</code>

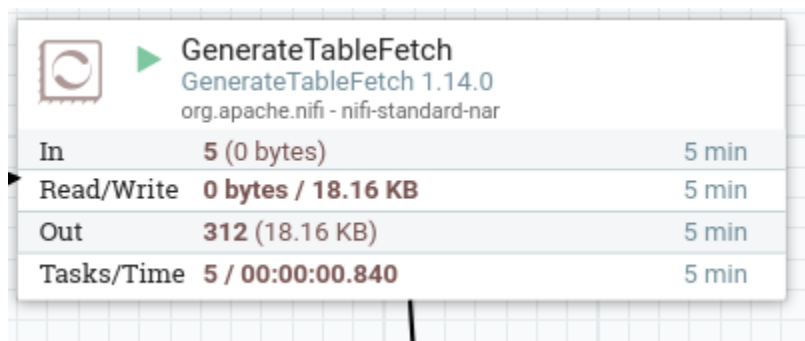
Name	Type	Bundle	State	Scope
 AvroRecordSetWriter	AvroRecordSetWriter 1.14...	org.apache.nifi - nifi-recor...	 Enabled	Ecommerce
 DBCPConnectionPool	DBCPConnectionPool 1.1...	org.apache.nifi - nifi-dbc...	 Enabled	Ecommerce

Property	Value
Database Connection URL	 <code>jdbc:mysql://localhost:3306/ecommerce</code>
Database Driver Class Name	 <code>com.mysql.jdbc.Driver</code>
Database Driver Location(s)	 <code>/usr/local/hive/hive-3.1.2/lib/mysql-connector-java-5.1...</code>
Kerberos Credentials Service	 No value set
Kerberos Principal	 No value set
Kerberos Password	 No value set
Database User	 <code>student</code>
Password	 Sensitive value set
Max Wait Time	 <code>500 millis</code>
Max Total Connections	 <code>8</code>
Validation query	 No value set
Minimum Idle Connections	 <code>0</code>
Max Idle Connections	 <code>8</code>
Max Connection Lifetime	 <code>-1</code>

Dynamic Flow Generation: Generates individual **FlowFiles** for each table without hardcoding table names, attaching the table name as an attribute: `${db.table.name}`.

Scheduling & Efficiency: Configured with a periodic **Run Schedule** (e.g., 1 day) to trigger the ingestion process on a controlled frequency, preventing redundant database overload.

And enables automated schema expansion if new tables are added in MariaDB.



The screenshot shows the configuration for the 'GenerateTableFetch' processor. It includes a status bar with 'In', 'Read/Write', 'Out', and 'Tasks/Time' metrics. The 'In' metric shows 5 (0 bytes) and 5 min. The 'Read/Write' metric shows 0 bytes / 18.16 KB and 5 min. The 'Out' metric shows 312 (18.16 KB) and 5 min. The 'Tasks/Time' metric shows 5 / 00:00:00.840 and 5 min.

Metric	Value	Unit
In	5 (0 bytes)	5 min
Read/Write	0 bytes / 18.16 KB	5 min
Out	312 (18.16 KB)	5 min
Tasks/Time	5 / 00:00:00.840	5 min

- Calculates total record counts and primary key boundaries to construct partitioned SQL **SELECT** queries for each incoming table.

Property	Value
Database Connection Pooling Service	DBCConnectionPool
Database Type	Generic
Table Name	<code>\${db.table.name}</code>
Columns to Return	No value set
Maximum-value Columns	No value set
Max Wait Time	0 seconds
Partition Size	10000
Column for Value Partitioning	No value set
Additional WHERE clause	No value set
Custom ORDER BY Column	No value set
Output Empty FlowFile on Zero Results	false

- **Partitioning Mechanism:** Uses dynamic property `${db.table.name}` and a configurable Partition Size (10,000 records) to divide large database tables into manageable page chunks.
- **Sqoop Equivalence:** Serves as NiFi's equivalent to Sqoop's `--split-by` enabling multi-threaded parallel extraction downstream.
- **State Management:** Tracks maximum column values (State) to allow incremental loading and prevent fetching previously processed records in future runs.

ExecuteSQLRecord		
ExecuteSQLRecord 1.14.0		
org.apache.nifi - nifi-standard-nar		
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

- Executes the partitioned SQL queries against MariaDB and directly serializes the output into Avro format.

Property	Value
Database Connection Pooling Service	DBCPConnectionPool
SQL Pre-Query	No value set
SQL select query	No value set
SQL Post-Query	No value set
Max Wait Time	0 seconds
Record Writer	AvroRecordSetWriter
Normalize table/column names	false
Use Avro Logical Types	false

- On-the-Fly Conversion: Combines query execution and format conversion in a single step using **AvroRecordSetWriter**, avoiding intermediary disk writes and eliminating the need for a separate **ConvertRecord** processor.

Scheduling Strategy	Run Duration
Timer driven	00:00:00.000
Concurrent Tasks	Run Schedule
4	0 sec
Execution	
All nodes	

- High Throughput & Parallelism: Configured with multiple Concurrent Tasks (e.g., 4 threads) to execute incoming queries in parallel, replicating Sqoop's multi-mapper performance.

 UpdateAttribute UpdateAttribute 1.14.0 org.apache.nifi - nifi-update-attribute-nar		
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

- Dynamically injects and modifies metadata attributes for each FlowFile prior to landing in HDFS.

Property	Value	
Delete Attributes Expression		No value set
Store State		Do not store state
Stateful Variables Initial Value		No value set
Cache Value Lookup Cache Size		100
filename		\${db.table.name}_\${now():format('yyyyMMdd_HH:mm:ss')}_\${fragment.id...
path		/staging_zone/ecommerce/\${db.table.name}

- **Dynamic HDFS Routing:** Constructs the destination directory path dynamically using `/staging_zone/ecommerce/${db.table.name}`
- **Unique Naming Strategy:** `filename` attribute using timestamp expressions and fragment identifiers (`${db.table.name}_${now():format('yyyyMMdd_HH:mm:ss')}_${fragment.id...}` identifier).`avro`).
- **Conflict Prevention:** Ensures that parallel files produced by concurrent mappers never overwrite each other in HDFS.



PutHDFS

PutHDFS 1.14.0

org.apache.nifi - nifi-hadoop-nar

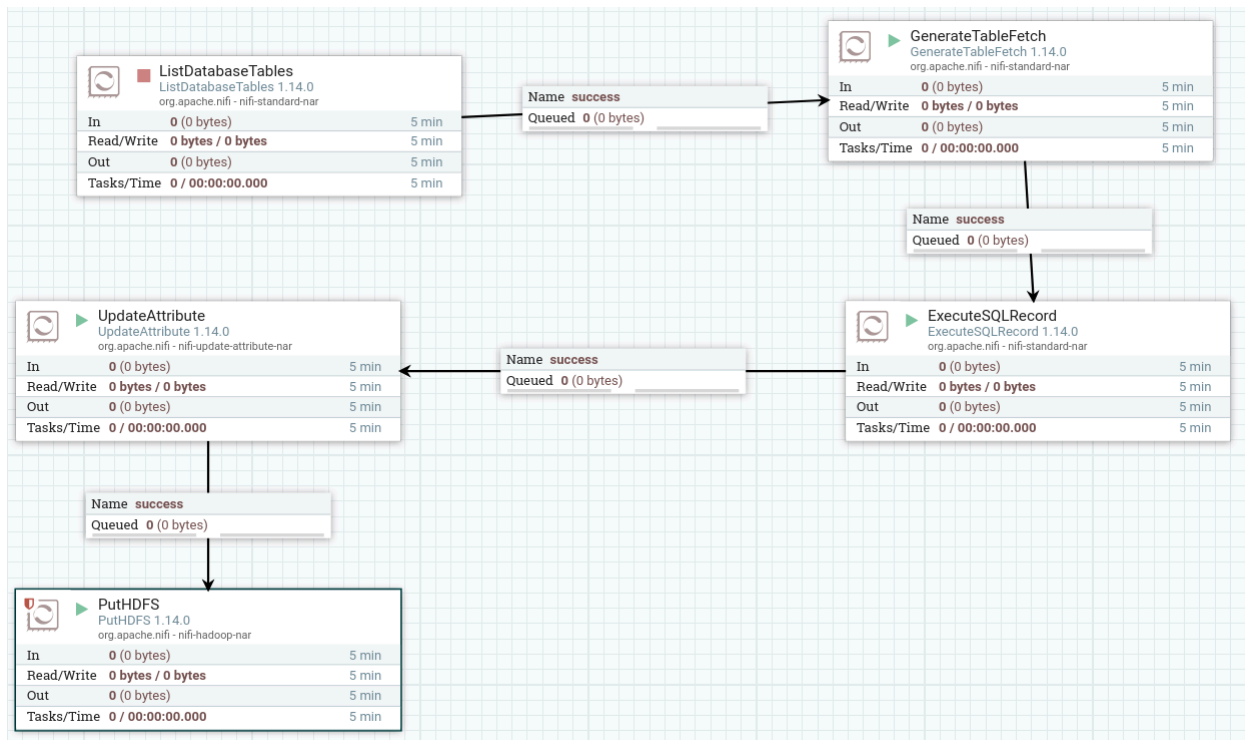
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Writes incoming binary FlowFiles into the Apache Hadoop Distributed File System

Property	Value
Hadoop Configuration Resources	 /home/hadoop/hadoop/etc/hadoop/core-site.xml/ho...
Kerberos Credentials Service	 No value set
Kerberos Principal	 No value set
Kerberos Keytab	 No value set
Kerberos Password	 No value set
Kerberos Relogin Period	 4 hours
Additional Classpath Resources	 No value set
Directory	 \${path}
Conflict Resolution Strategy	 replace
Block Size	 No value set
IO Buffer Size	 No value set
Replication	 No value set

Directory (`${path}`**):** Reads the dynamic output directory generated in **UpdateAttribute** (**`/staging_zone/ecommerce/${db.table.name}`**), routing each table to its target HDFS directory.

Conflict Resolution Strategy: Handles file collision logic in HDFS. Set to **replace**



Verifying the main HDFS staging area /staging_zone/ecommerce, showing that all 5 extracted database tables (customers, order_items, orders, payments, products) were successfully created as dedicated directories

```
[student@localhost ~]$ hdfs dfs -ls /staging_zone/ecommerce
Found 5 items
drwxr-xr-x - root supergroup          0 2026-09-07 02:13 /staging_zone/ecommerce/customers
drwxr-xr-x - root supergroup          0 2026-09-07 02:14 /staging_zone/ecommerce/order_items
drwxr-xr-x - root supergroup          0 2026-09-07 02:14 /staging_zone/ecommerce/orders
drwxr-xr-x - root supergroup          0 2026-09-07 02:14 /staging_zone/ecommerce/payments
drwxr-xr-x - root supergroup          0 2026-09-07 02:14 /staging_zone/ecommerce/products
```

The **customers** table yielded 1 Avro file because its total record count fell within our configured partition size limit (e.g., 10,000 records per chunk)

```
[student@localhost ~]$ hdfs dfs -ls /staging_zone/ecommerce/customers
Found 1 items
-rw-r--r-- 1 root supergroup 915487 2026-09-07 02:13 /staging_zone/ecommerce/customers/customers_20260907_021329_07dbb219-2c5f-4838-bdeb-59c48ec965a6.avro
```

The **orders** table was split into 2 Avro partition files, reflecting the parallel fetch logic based on table size boundaries

```
[student@localhost ~]$ hdfs dfs -ls /staging_zone/ecommerce/orders
Found 2 items
-rw-r--r-- 1 root supergroup 474634 2026-09-07 02:14 /staging_zone/ecommerce/orders/orders_20260907_021338_07893bd1-5c96-403a-b850-2fbc3715e18b.avro
-rw-r--r-- 1 root supergroup 474180 2026-09-07 02:14 /staging_zone/ecommerce/orders/orders_20260907_021339_07893bd1-5c96-403a-b850-2fbc3715e18b.avro
```

As the largest table in the database, **order_items** was dynamically chunked into 10 separate Avro partition files via GenerateTableFetch to enable optimized parallel ingestion and balanced processing

```
[student@localhost ~]$ hdfs dfs -ls /staging_zone/ecommerce/order_items
Found 10 items
-rw-r--r-- 1 root supergroup 270922 2026-09-07 02:13 /staging_zone/ecommerce/order_items/order_items_20260907_021329_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 281136 2026-09-07 02:13 /staging_zone/ecommerce/order_items/order_items_20260907_021330_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 281211 2026-09-07 02:13 /staging_zone/ecommerce/order_items/order_items_20260907_021331_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 291282 2026-09-07 02:13 /staging_zone/ecommerce/order_items/order_items_20260907_021332_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 291222 2026-09-07 02:14 /staging_zone/ecommerce/order_items/order_items_20260907_021333_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 291134 2026-09-07 02:14 /staging_zone/ecommerce/order_items/order_items_20260907_021334_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 291146 2026-09-07 02:14 /staging_zone/ecommerce/order_items/order_items_20260907_021335_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 291107 2026-09-07 02:14 /staging_zone/ecommerce/order_items/order_items_20260907_021336_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 291196 2026-09-07 02:14 /staging_zone/ecommerce/order_items/order_items_20260907_021337_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
-rw-r--r-- 1 root supergroup 291303 2026-09-07 02:14 /staging_zone/ecommerce/order_items/order_items_20260907_021338_96c262cb-1a6c-47ff-951f-e6e1761360a2.avro
```

payments was partitioned into 2 Avro files, while products fits into 1 Avro file, confirming that dynamic naming and folder creation per table worked as expected.

```
[student@localhost ~]$ hdfs dfs -ls /staging_zone/ecommerce/payments
Found 2 items
-rw-r--r-- 1 root supergroup 530242 2026-09-07 02:14 /staging_zone/ecommerce/payments/payments_20260907_021339_f1d4de89-1d26-4380-801f-2431d5d7555d.avro
-rw-r--r-- 1 root supergroup 529870 2026-09-07 02:14 /staging_zone/ecommerce/payments/payments_20260907_021340_f1d4de89-1d26-4380-801f-2431d5d7555d.avro
[student@localhost ~]$ hdfs dfs -ls /staging_zone/ecommerce/products
Found 1 items
-rw-r--r-- 1 root supergroup 572394 2026-09-07 02:14 /staging_zone/ecommerce/products/products_20260907_021340_237b419e-9948-4c6c-8958-db616fb8dfc1.avro
[student@localhost ~]$
```



1. The Problem: Implicit Local Spark Context



When initially launching the environment using the `pyspark` CLI command, PySpark automatically initializes a default background `SparkContext` configured to run locally (`local[*]`). As a result, calling `.master("yarn")` inside the notebook script was overridden by the existing active session via `.getOrCreate()`. This kept the job running in standalone local mode and prevented it from registering on the YARN Resource Manager cluster

```
In [3]: from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("EcommerceETL")
    .master("yarn")
    .enableHiveSupport()
    .getOrCreate()
)

print(spark.sparkContext.master)

local[*]
```

2. The Solution & YARN Resource Management

To resolve this issue, Jupyter Notebook was launched independently via the `jupyter notebook` command without pre-initializing a Spark context.

This allowed the script to explicitly define `.master("yarn")` during session instantiation. By offloading job execution to **YARN**, YARN assumes full responsibility as the cluster resource manager—dynamically allocating memory, CPU cores, and managing distributed execution across the Hadoop cluster rather than relying on local system resources.

```
In [2]: from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("EcommerceETL")
    .master("yarn")
    .enableHiveSupport()
    .getOrCreate()
)

print(spark.sparkContext.master)

2026-09-07 02:57:22,773 WARN util.Utils: Your hostname, localhost.localdomain resolves to a loopback address: 127.0.0.1; using 10.0.2.15 instead (on interface enp0s3)
2026-09-07 02:57:22,774 WARN util.Utils: Set SPARK_LOCAL_IP if you need to bind to another address
2026-09-07 02:57:23,033 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
2026-09-07 02:57:24,690 WARN yarn.Client: Neither spark.yarn.jars nor spark.yarn.archive is set, falling back to uploading libraries under SPARK_HOME.

yarn
```

3. Verification via the YARN Web UI.



All Application

Cluster

About Nodes

Node Labels

Applications

NEW

NEW SAVING

SUBMITTED

ACCEPTED

RUNNING

FINISHED

FAILED

KILLED

Scheduler

Tools

Cluster Metrics

Apps Submitted

Apps Pending

Apps Running

Apps Completed

Containers Running

Used Resources

1

1

0

0

0

<memory:0 B, vCores:0>

Cluster Nodes Metrics

Active Nodes

Decommissioning Nodes

Decommissioned Nodes

1

0

0

0

Scheduler Metrics

Scheduler Type

Scheduling Resource Type

Minimum Allocation

Capacity Scheduler

[memory-mb (unit=Mi), vcores]

<memory:1024, vCores:1>

<memory:8192, vCores:1>

Show 20 entries

ID

User

Name

Application Type

Application Tags

Queue

Application Priority

StartTime

LaunchTime

FinishTime

State

FinalStatus

application_1788708101114_0001

student

EcommerceETL

SPARK

default

0

Mon Sep 7 02:57:29 +0900 2026

N/A

N/A

ACCEPTED

UNDEFINED

I change port of jupyter because Hue & jupyter appear on same port

```
File Edit View Bookmarks Settings Help
[student@localhost ~]$ jupyter notebook --port 8889
```

And then upload and start to run 2.transformation_script file



the problem i face on that

The error because the instructor saved files as CSV and that code who 's write make to read data on HDFS as csv but i am change on my nifi and make data save as AVRO so i change that code

```
: customers = (
    spark.read
    .option("header", "false")
    .option("inferSchema", "true")
    .option("sep", ",")
    .csv(f"{RAW_PATH}/customers")
)

customers = customers.toDF(
    "customer_id",
    "first_name",
    "last_name",
    "email",
    "country",
    "city",
    "signup_date"
)

customers.printSchema()
customers.show(5)
```

```
-----
IllegalArgumentExceptio                               Tra
ast)
/tmp/ipykernel_27567/1880053347.py in <module>
    7 )
    8
```



```
customers = (
    spark.read
    .format("avro")
    .load(f"{RAW_PATH}/customers")
)

customers.printSchema()
customers.show(5)
```

```
root
|-- customer_id: string (nullable = true)
|-- first_name: string (nullable = true)
|-- last_name: string (nullable = true)
|-- email: string (nullable = true)
|-- country: string (nullable = true)
|-- city: string (nullable = true)
|-- signup_date: string (nullable = true)
```

```
+-----+-----+-----+-----+-----+-----+
|customer_id|first_name|last_name|email|country|
city|signup_date|
+-----+-----+-----+-----+-----+-----+
|90001|Stephen|Welch|stephen.welch9000...|Saudi Arabia|
Samanthaton|1755907200000|
|90002|Derrick|Villa|derrick.villa9000...|Qatar|
New David|1695945600000|
|90003|Juan|Robertson|juan.robertson900...|UAE| J
acksonmouth|1641859200000|
|90004|Nicole|Hernandez|nicole.hernandez9...|Kuwait|Po
rt Kurtside|1728604800000|
|90005|Nathan|Peters|nathan.peters9000...|Jordan| S
outh Mariah|1753142400000|
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Then i change all cell to make read as Avro format

```
products = (
    spark.read
    .option("header", "false")
    .option("inferSchema", "true")
    .option("sep", ",")
    .csv(f"{RAW_PATH}/products")
)

products = products.toDF(
    "product_id",
    "product_name",
    "category",
    "price",
    "stock_quantity"
)

products.printSchema()
products.show(5)
```

```
-----
IllegalArgumentExcep      Traceback (most recent call
last)
/tmp/ipykernel_27567/4290509343.py in <module>
      7 )
      8
----> 9 products = products.toDF(
     10     "product_id",
     11     "product_name",

/usr/local/spark3/spark-3.1.2-bin-hadoop3.2/python/pyspark/sql/datafra
me.py in toDF(self, *cols)
    2532         [Row(f1=2, f2='Alice'), Row(f1=5, f2='Bob')]
    2533         """
```

```
products = (
    spark.read
    .format("avro")
    .load(f"{RAW_PATH}/products")
)

products.printSchema()
products.show(5)
```

```
root
|-- product_id: string (nullable = true)
|-- product name: string (nullable = true)
|-- category: string (nullable = true)
|-- price: decimal(10,2) (nullable = true)
|-- stock_quantity: integer (nullable = true)
```

[Stage 7: >+ 1) / 1]

product_id	product_name	category	price	stock
1	Exclusive composi...	Gaming	1978.97	
2	Mandatory dedicat...	Furniture	1511.46	
3	Secured radical s...	Gaming	1542.36	
4	Re-contextualized...	Beauty	1647.17	
5	Total foreground ...	Sports	1024.78	

only showing top 5 rows

```
orders = (
    spark.read
    .option("header", "false")
    .option("inferSchema", "true")
    .option("sep", ",")
    .csv(f"{RAW_PATH}/orders")
)

orders = orders.toDF(
    "order_id",
    "customer_id",
    "order_date",
    "status",
    "shipping_country"
)

orders.printSchema()
orders.show(5)
```

```
-----
IllegalArgumentExcep      Traceback (most recent call
last)
/tmp/ipykernel_27567/1322267898.py in <module>
      7 )
      8
----> 9 orders = orders.toDF(
     10     "order_id",
     11     "customer_id",

/usr/local/spark3/spark-3.1.2-bin-hadoop3.2/python/pyspark/sql/datafra
me.py in toDF(self, *cols)
    2532         [Row(f1=2, f2='Alice'), Row(f1=5, f2='Bob')]
    2533         """
-> 2534         jdf = self._jdf.toDF(self._jseq(cols))
    2535         return DataFrame(jdf, self.sql_ctx)
    2536
```

```
orders = (
    spark.read
    .format("avro")
    .load(f"{RAW_PATH}/orders")
)

orders.printSchema()
orders.show(5)
```

```
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- order_date: string (nullable = true)
|-- status: string (nullable = true)
|-- shipping_country: string (nullable = true)
```

order_id	customer_id	order_date	status	shipping_country
290001	35598	1730270732000	pending	UAE
290002	43295	1756813950000	pending	Kuwait
290003	22819	1752208660000	completed	Jordan
290004	38887	1775486570000	completed	Saudi Arabia
290005	45282	1749476990000	cancelled	Qatar

only showing top 5 rows

```
order_items = (
    spark.read
    .option("header", "false")
    .option("inferSchema", "true")
    .option("sep", ",")
    .csv(f"{RAW_PATH}/order_items")
)
```

```
order_items = order_items.toDF(
    "order_item_id",
    "order_id",
    "product_id",
    "quantity",
    "unit_price"
)
```

```
order_items.printSchema()
order_items.show(5)
```

 IllegalArgumentException Traceback (most recent call last)

/tmp/ipykernel_27567/1607037341.py in <module>

```
7 )
8
----> 9 order_items = order_items.toDF(
10     "order_item_id",
11     "order_id",
```

/usr/local/spark3/spark-3.1.2-bin-hadoop3.2/python/pyspark/sql/dataframe.py in toDF(self, *cols)

```
2532 [Row(f1=2, f2='Alice'), Row(f1=5, f2='Bob')]
2533 """
-> 2534 idf = self.idf.toDF(self.iseq(cols))
```

```
order_items = (
    spark.read
    .format("avro")
    .load(f"{RAW_PATH}/order_items")
)
```

```
order_items.printSchema()
order_items.show(5)
```

root

```
-- order_item_id: string (nullable = true)
-- order_id: string (nullable = true)
-- product_id: string (nullable = true)
-- quantity: integer (nullable = true)
-- unit_price: decimal(10,2) (nullable = true)
```

order_item_id	order_id	product_id	quantity	unit_price
1990001	347626	7008	1	1289.14
1990002	82704	19722	1	348.96
1990003	273492	3714	2	1088.12
1990004	110082	9241	4	431.53
1990005	192387	17972	2	88.82

only showing top 5 rows


```
payments = (
    spark.read
    .option("header", "false")
    .option("inferSchema", "true")
    .option("sep", ",")
    .csv(f"{RAW_PATH}/payments")
)
```

```
payments = payments.toDF(
    "payment_id",
    "order_id",
    "payment_method",
    "payment_status",
    "payment_date",
    "amount"
)
```

```
payments.printSchema()
payments.show(5)
```

```
-----
IllegalArgumentExcpion                                Traceback (most recent ca
last)
/tmp/ipykernel_27567/1385777034.py in <module>
      7 )
      8
----> 9 payments = payments.toDF(
     10     "payment_id",
     11     "order_id",

/usr/local/spark3/spark-3.1.2-bin-hadoop3.2/python/pyspark/sql/data
me.py in toDF(self, *cols)
    2532         [Row(f1=2, f2='Alice'), Row(f1=5, f2='Bob')]
    2533         """
-> 2534         jdf = self._jdf.toDF(self._jseq(cols))
    2535         return DataFrame(jdf, self.sql_ctx)
    2536
```

```
payments = (
    spark.read
    .format("avro")
    .load(f"{RAW_PATH}/payments")
)
```

```
payments.printSchema()
payments.show(5)
```

```
root
 |-- payment_id: string (nullable = true)
 |-- order_id: string (nullable = true)
 |-- payment_method: string (nullable = true)
 |-- payment_status: string (nullable = true)
 |-- payment_date: string (nullable = true)
 |-- amount: decimal(10,2) (nullable = true)
```

```
+-----+-----+-----+-----+-----+
--+
|payment_id|order_id|payment_method|payment_status| payment_date| am
nt|
+-----+-----+-----+-----+-----+
--+
|  280001|  280001|          cash|          failed|1779224996000|483
68|
|  280002|  280002| bank_transfer|          paid|1726337610000|337
63|
|  280003|  280003| bank_transfer|          paid|1755193694000|436
58|
|  280004|  280004| bank_transfer|        refunded|1758169336000|291
80|
|  280005|  280005|         paypal|          paid|1726872813000|437
23|
+-----+-----+-----+-----+-----+
--+
only showing top 5 rows
```

Data Quality Check & Deduplication Verification

To ensure data integrity across the ingestion pipeline, primary key deduplication was performed on all Avro DataFrames using `dropDuplicates()`. The final record counts confirmed successful dataset verification and readiness for transformation:

- Customers: 10,000 unique records
- Products: 10,000 unique records
- Orders: 20,000 unique records
- Order Items: 100,000 unique records
- Payments: 20,000 unique records

All tables have been cleaned of duplicates and loaded into memory as optimized Spark DataFrames.

```
customers = customers.dropDuplicates(["customer_id"])

products = products.dropDuplicates(["product_id"])

orders = orders.dropDuplicates(["order_id"])

order_items = order_items.dropDuplicates(["order_item_id"])

payments = payments.dropDuplicates(["payment_id"])

print("Customers:", customers.count())
print("Products:", products.count())
print("Orders:", orders.count())
print("Order Items:", order_items.count())
print("Payments:", payments.count())
```

Customers: 10000

Products: 10000

Orders: 20000

Order Items: 100000

[Stage 30:=====>(199
1) / 200]

Payments: 20000



Here another Problem



The standard CSV date parser failed on Avro's numeric Unix Epoch Timestamps, causing a silent failure where all dates became **null**.

The Root Cause: Avro stores timestamps natively as numeric milliseconds rather than plain text strings.

The Solution: We divided the millisecond values by \$1,000\$ and applied a direct **.cast("timestamp")**, preserving the complete date and time hierarchy.

```
orders = orders.withColumn(  
    "order_date",  
    to_date("order_date", "yyyy-MM-dd HH:mm:ss")  
)
```

```
orders.printSchema()
```

```
root  
|-- order_id: string (nullable = true)  
|-- customer_id: string (nullable = true)  
|-- order_date: date (nullable = true)  
|-- status: string (nullable = true)  
|-- shipping_country: string (nullable = true)
```

```
orders.show(5)
```

order_id	customer_id	order_date	status	shipping_country
290183	89552	null	pending	Saudi Arabia
290377	81586	null	completed	Egypt
290468	43586	null	pending	Saudi Arabia
290546	32282	null	completed	Saudi Arabia
290844	36052	null	completed	Kuwait

only showing top 5 rows

```
from pyspark.sql.functions import col  
orders = orders.withColumn(  
    "order_date",  
    (col("order_date") / 1000).cast("timestamp")  
)
```

```
orders.printSchema()  
orders.show(5, truncate=False)
```

```
root  
|-- order_id: string (nullable = true)  
|-- customer_id: string (nullable = true)  
|-- order_date: timestamp (nullable = true)  
|-- status: string (nullable = true)  
|-- shipping_country: string (nullable = true)
```

order_id	customer_id	order_date	status	shipping_country
290001	35598	2024-10-30 15:45:32	pending	UAE
290002	43295	2025-09-02 20:52:30	pending	Kuwait
290003	22819	2025-07-11 13:37:40	completed	Jordan
290004	38887	2026-04-06 23:42:50	completed	Saudi Arabia
290005	45282	2025-06-09 22:49:50	cancelled	Qatar

only showing top 5 rows

```
orders.printSchema()
```

```
root  
|-- order_id: string (nullable = true)  
|-- customer_id: string (nullable = true)  
|-- order_date: timestamp (nullable = true)  
|-- status: string (nullable = true)  
|-- shipping_country: string (nullable = true)
```

Here all tables saved on hive and ready to analysis

```
: spark.sql(f"DROP TABLE IF EXISTS {HIVE_DB}.fact_sales")
spark.sql(f"DROP TABLE IF EXISTS {HIVE_DB}.dim_product")

fact_sales.write \
    .mode("overwrite") \
    .format("orc") \
    .partitionBy("year", "month") \
    .saveAsTable(f"{HIVE_DB}.fact_sales")

dim_product.write \
    .mode("overwrite") \
    .format("orc") \
    .saveAsTable(f"{HIVE_DB}.dim_product")
```

```
: dim_customer.write \
    .mode("overwrite") \
    .format("parquet") \
    .option("compression", "snappy") \
    .saveAsTable(
        f"{HIVE_DB}.dim_customer"
    )
```

```
: daily_sales.write \
    .mode("overwrite") \
    .format("parquet") \
    .option("compression", "snappy") \
    .saveAsTable(
        f"{HIVE_DB}.daily_sales"
    )
```

```
: spark.stop()
```

Some important analysis using matplotlib to make bar chart analysis every year with total revenue

```
import matplotlib.pyplot as plt
import seaborn as sns

revenue_by_year = spark.sql(f"""
    SELECT year, SUM(total_sales) AS total_revenue
    FROM {HIVE_DB}.daily_sales
    GROUP BY year
    ORDER BY year
""").toPandas()

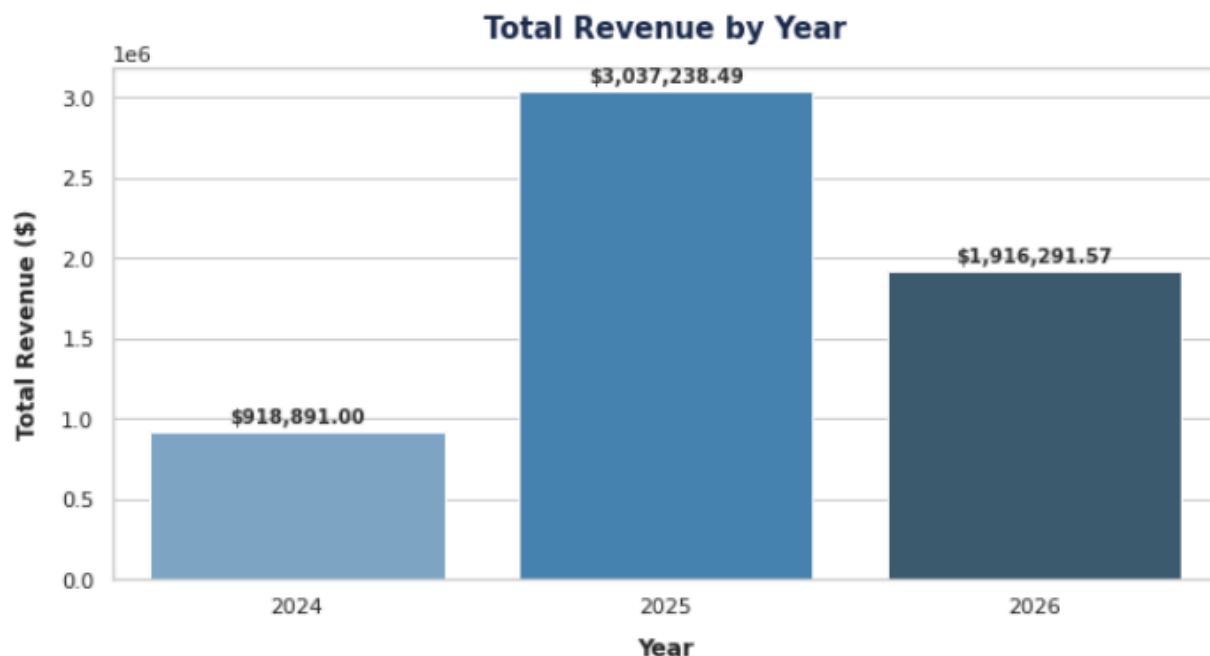
plt.figure(figsize=(9, 5))
sns.set_theme(style="whitegrid")

ax = sns.barplot(
    x="year",
    y="total_revenue",
    data=revenue_by_year,
    palette="Blues_d"
)

plt.title("Total Revenue by Year", fontsize=15, fontweight='bold', pad=15, color='#1A2B4C')
plt.xlabel("Year", fontsize=12, fontweight='bold', labelpad=10)
plt.ylabel("Total Revenue ($)", fontsize=12, fontweight='bold', labelpad=10)

for p in ax.patches:
    height = p.get_height()
    ax.annotate(
        f"${height:,.2f}",
        (p.get_x() + p.get_width() / 2., height),
        ha='center', va='center',
        xytext=(0, 8),
        textcoords='offset points',
        fontsize=10, fontweight='bold', color='#333333'
    )

plt.tight_layout()
plt.show()
```



```

category_sales = spark.sql(f"""
    SELECT category, SUM(sales_amount) AS total_revenue
    FROM {HIVE_DB}.fact_sales
    GROUP BY category
    ORDER BY total_revenue DESC
""").toPandas()

plt.figure(figsize=(10, 6))
sns.set_theme(style="whitegrid")

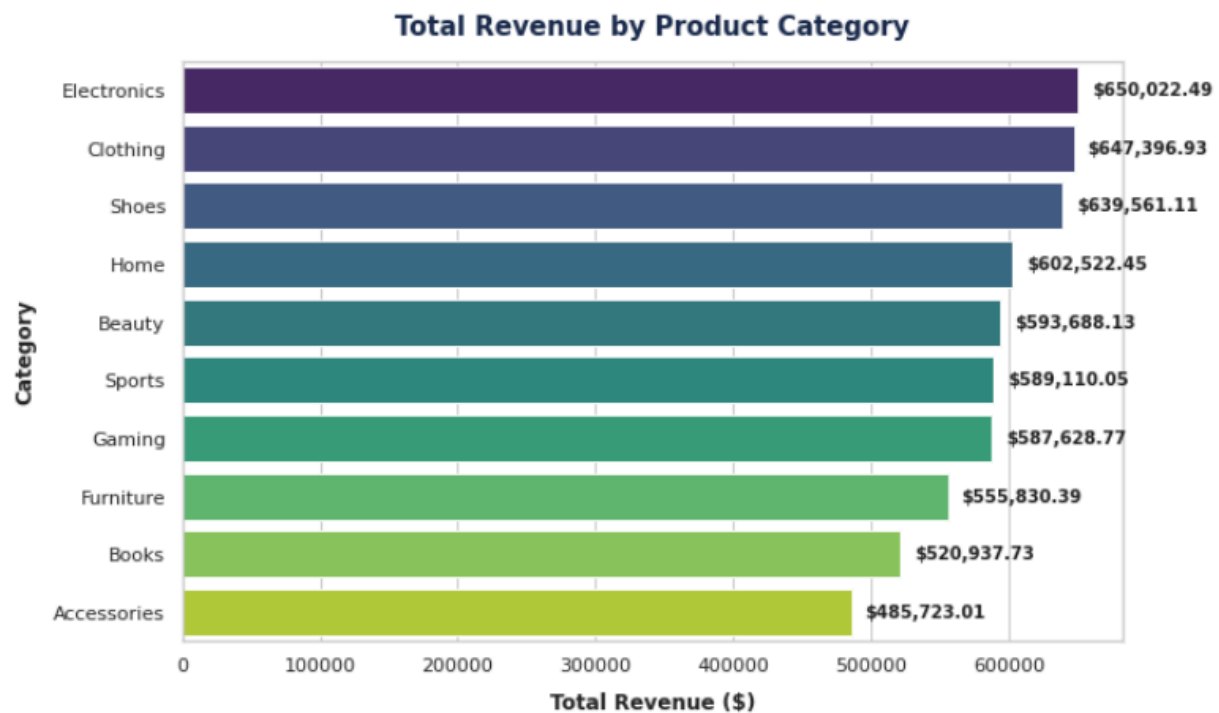
ax = sns.barplot(
    x="total_revenue",
    y="category",
    data=category_sales,
    hue="category",
    palette="viridis",
    legend=False
)

plt.title("Total Revenue by Product Category", fontsize=15, fontweight='bold', pad=15, color='#1A2B4C')
plt.xlabel("Total Revenue ($) ", fontsize=12, fontweight='bold', labelpad=10)
plt.ylabel("Category", fontsize=12, fontweight='bold', labelpad=10)

for p in ax.patches:
    width = p.get_width()
    ax.annotate(
        f"${width:,.2f}",
        (width, p.get_y() + p.get_height() / 2.),
        ha='left', va='center',
        xytext=(8, 0),
        textcoords='offset points',
        fontsize=10, fontweight='bold', color='#222222'
    )

plt.tight_layout()
plt.show()

```



<  ecommerce_dw_test

Tables

(4) + ↺

Filter...

-  daily_sales
-  dim_customer
-  dim_product
-  fact_sales

To transform all internal tables to external tables

0.26s ecommerce_dw_test ▼ ⚙

```
1
2 ALTER TABLE ecommerce_dw_test.fact_sales SET TBLPROPERTIES('EXTERNAL'='TRUE');
3
4 DESCRIBE FORMATTED ecommerce_dw_test.fact_sales;
5
6 ALTER TABLE ecommerce_dw_test.daily_sales SET TBLPROPERTIES('EXTERNAL'='TRUE');
7
8 ALTER TABLE ecommerce_dw_test.dim_customer SET TBLPROPERTIES('EXTERNAL'='TRUE');
9
10 ALTER TABLE ecommerce_dw_test.dim_product SET TBLPROPERTIES('EXTERNAL'='TRUE');
```

```
INFO : Executing command(queryId=root_20260907062342_6a609448-b2af-4910-8234-01487f6a018e): DESCRIBE FORMATTED ecommerce_dw_test.fact_sales
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=root_20260907062342_6a609448-b2af-4910-8234-01487f6a018e); Time taken: 0.054 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager
```

Query History

Saved Queries

Results (57)

	col_name	data_type	comment
1	# col_name	data_type	comment
2	order_item_id	string	
3	order_id	string	
4	customer_id	string	
5	product_id	string	
6	quantity	int	
7	unit_price	decimal(10,2)	
8	sales_amount	decimal(21,2)	
9	order_date	date	
10	status	string	

Here some important analysis i do

14
15
16
17
18

INFO : Executing command(queryId=root_20260907062514_159c2554-0c89-40f1-9712-188a6f76255f) :
SELECT * FROM ecommerce_dw_test.fact_sales LIMIT 10
INFO : Completed executing command(queryId=root_20260907062514_159c2554-0c89-40f1-9712-188a6f76255f); Time taken: 0.0 seconds
INFO : OK
INFO : Concurrency mode is disabled, not creating a lock manager

Query HistorySaved QueriesResults (10)

	fact_sales.order_item_id	fact_sales.order_id	fact_sales.customer_id	fact_sales.product_id	fact_sales.quantity	fact_sales.unit_price
1	566812	486794	64633	9495	5	1495.23
2	565689	299635	42433	2330	4	1246.45
3	1949755	299774	5283	2312	4	1644.45
4	895428	299680	81425	7527	5	1350.09
5	896532	485111	33645	9086	3	1979.62
6	1482974	481072	75932	2559	2	379.93
7	1997111	297330	56706	7550	3	397.00
8	1999316	487087	37792	6680	2	546.14

7m, 42s ecommerce_dw_test

37
38
39
40
41
42
43
44

SELECT year, month, day, total_sales, total_orders
FROM ecommerce_dw_test.daily_sales
ORDER BY total_sales DESC
LIMIT 5;

INFO : Submitting tokens for job: job_1788708101114_0008
INFO : Executing with tokens: []
INFO : The url to track the job: http://localhost:8088/proxy/application_1788708101114_0008/
INFO : Starting Job = job_1788708101114_0008, Tracking URL = http://localhost:8088/proxy/application_1788708101114_0008/
INFO : Kill Command = /home/hadoop/hadoop/bin/mapred job -kill job_1788708101114_0008

job_1788708101114_0008

Query HistorySaved Queries

8 minutes ago

SELECT COUNT(*) FROM ecommerce_dw_test.fact_sales WHERE order_id IS NULL OR customer_id IS NULL

9 minutes ago

SELECT * FROM ecommerce_dw_test.fact_sales LIMIT 10

11 minutes ago

DESCRIBE FORMATTED ecommerce_dw_test.fact_sales

11 minutes ago

ALTER TABLE ecommerce_dw_test.fact_sales SET TBLPROPERTIES('EXTERNAL'='TRUE')

14 minutes ago

SELECT * FROM ecommerce_dw_test.fact_sales WHERE unit_price <= 0 OR quantity <= 0 LIMIT 5

16 minutes ago

SELECT COUNT(*) FROM ecommerce_dw_test.fact_sales WHERE order_id IS NULL OR customer_id IS NULL

16 minutes ago

SELECT year, month, SUM(total_sales) AS monthly_revenue FROM ecommerce_dw_test.daily_sales GROUP BY year, month ORDER BY year, month

18 minutes ago

SELECT COUNT(*) FROM ecommerce_dw_test.fact_sales WHERE order_id IS NULL OR customer_id IS NULL

11.88s ecommerce_dw_test ▾ ⚙

```
44  
45  
46 SELECT category, SUM(sales_amount) AS total_revenue, SUM(quantity) AS total_units_sold  
47 FROM ecommerce_dw_test.fact_sales  
48 GROUP BY category  
49 ORDER BY total_revenue DESC;  
50
```

INFO : Submitting tokens for job: job_1788708101114_0009
INFO : Executing with tokens: []
INFO : The url to track the job: http://localhost:8088/proxy/application_1788708101114_0009/
INFO : Starting Job = job_1788708101114_0009, Tracking URL = http://localhost:8088/proxy/application_1788708101114_0009/
INFO : Kill Command = /home/hadoop/hadoop/bin/mapred job -kill job_1788708101114_0009

[job_1788708101114_0009](#)

Query History

Saved Queries

a few seconds ago ➔	SELECT category, SUM(sales_amount) AS total_revenue, SUM(quantity) AS total_units_sold FROM ecommerce_dw_test.fact_sales GROUP BY category ORDER BY total_revenue DESC
9 minutes ago !	SELECT COUNT(*) FROM ecommerce_dw_test.fact_sales WHERE order_id IS NULL OR customer_id IS NULL
11 minutes ago ⚙	SELECT * FROM ecommerce_dw_test.fact_sales LIMIT 10
12 minutes ago ⚙	DESCRIBE FORMATTED ecommerce_dw_test.fact_sales
12 minutes ago ⚙	ALTER TABLE ecommerce_dw_test.fact_sales SET TBLPROPERTIES('EXTERNAL'='TRUE')
16 minutes ago ⚙	SELECT * FROM ecommerce_dw_test.fact_sales WHERE unit_price <= 0 OR quantity <= 0 LIMIT 5

2.76s ecommerce_dw_test ▾ ⚙

```
60 LIMIT 5;  
61  
62  
63 SELECT status, COUNT(*) AS status_count  
64 FROM ecommerce_dw_test.fact_sales  
65 GROUP BY status;  
66
```

INFO : Submitting tokens for job: job_1788708101114_0010
INFO : Executing with tokens: []
INFO : The url to track the job: http://localhost:8088/proxy/application_1788708101114_0010/
INFO : Starting Job = job_1788708101114_0010, Tracking URL = http://localhost:8088/proxy/application_1788708101114_0010/
INFO : Kill Command = /home/hadoop/hadoop/bin/mapred job -kill job_1788708101114_0010

[job_1788708101114_0010](#)

Query History

Saved Queries

a few seconds ago ➔	SELECT status, COUNT(*) AS status_count FROM ecommerce_dw_test.fact_sales GROUP BY status
a few seconds ago ➔	SELECT category, SUM(sales_amount) AS total_revenue, SUM(quantity) AS total_units_sold FROM ecommerce_dw_test.fact_sales GROUP BY category ORDER BY total_revenue DESC
10 minutes ago !	SELECT COUNT(*) FROM ecommerce_dw_test.fact_sales WHERE order_id IS NULL OR customer_id IS NULL
11 minutes ago ⚙	SELECT * FROM ecommerce_dw_test.fact_sales LIMIT 10